

Containers and semantic layout

Once a page has more than a heading and a few paragraphs, you start needing ways to group elements together. Group a logo and a navigation bar so they form a header. Group an article and a sidebar so they sit next to each other. Group three inputs so you can style them as one card. Almost every layout decision on a real page comes down to deciding which elements belong together.

HTML gives you two kinds of containers for this. **Generic containers** group elements but say nothing about their meaning. **Semantic containers** group elements and also describe what they are: a header, a navigation block, the main content of the page.

You should always reach for the semantic container first, and fall back to a generic one only when nothing more specific fits.

Block versus inline elements

Before containers make sense, it helps to know how HTML elements lay themselves out by default.

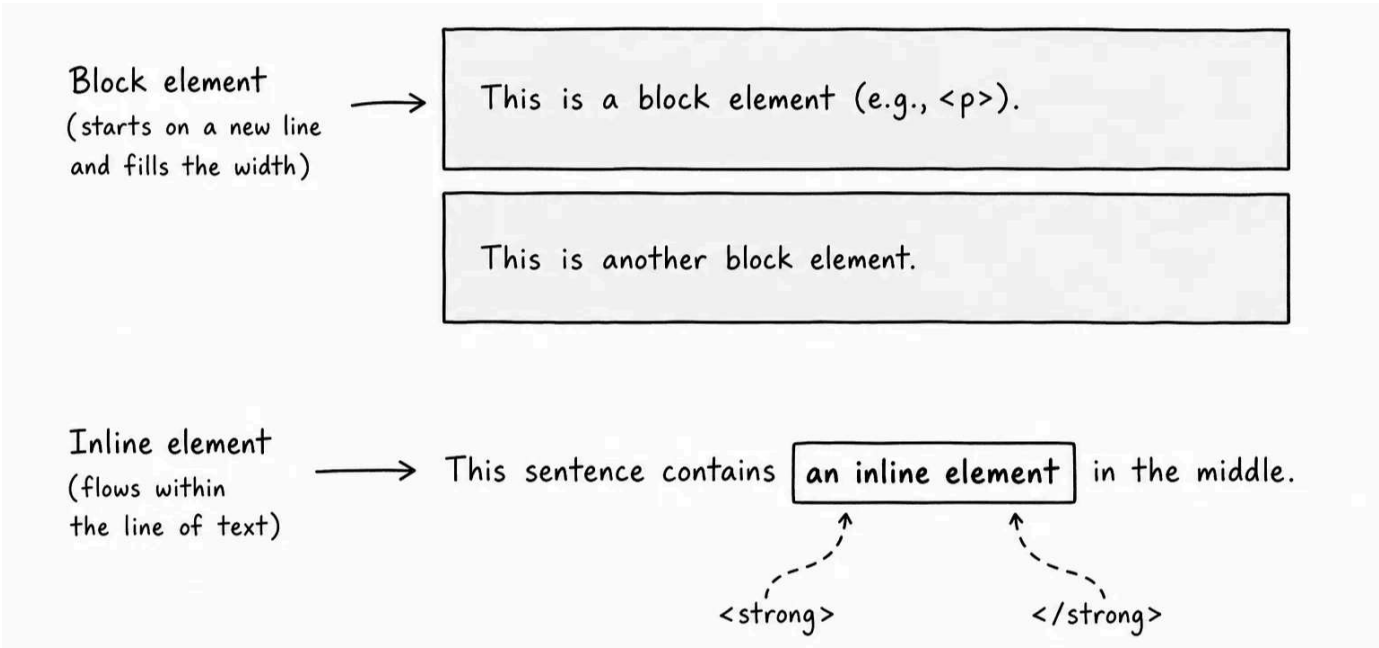
Most elements fall into two layout categories:

- **Block elements** start on a new line and stretch to fill the available width. Headings, paragraphs, lists, and most container elements behave this way.
- **Inline elements** sit inside a line of text without breaking it. `<a>` , `<em>` , `<strong>` , `<code>` , and `<img>` are inline by default.

html

```
<p>This paragraph is a block element.</p>
<p>This sentence contains <strong>an inline element</strong> in the middle.</p>
```

CSS can change any element's display mode, but the default is part of why HTML "just works" before any styling. Block elements form rows. Inline elements flow with the surrounding text.



Generic containers: div and span

When you just need to group elements with no implied meaning, HTML has two generic boxes.

`<div>` is a generic block container. It is usually used for larger chunks of content.

html

```
<div>
  <p>This paragraph and the next one are inside the same group.</p>
  <p>Anything CSS does to the div applies to both.</p>
</div>
```

`<span>` is a generic inline container. It groups a small piece of text inside a line.

html

```
<p>Hover over <span class="hint">this word</span> to see a tooltip.</p>
```

Neither element means anything on its own. They are there to give you a hook for CSS classes, JavaScript event handlers, or layout adjustments. The browser, the screen reader, and the search engine all treat them as "an element you wanted to group", no more and no less.

Generic containers are useful, but they are also the lazy default. The next sections are about why a more specific container is almost always a better choice.

When generic containers add up: div soup

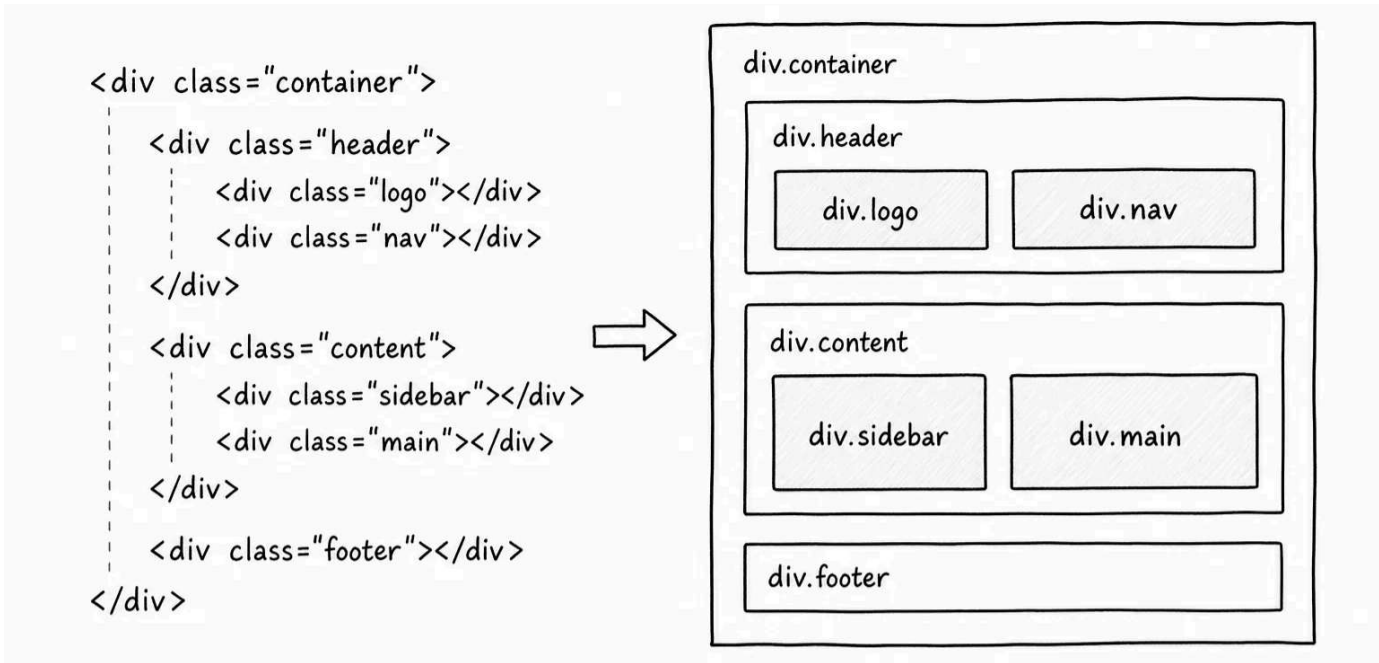
A page written entirely with `<div>` and `<span>` is sometimes called "div soup". Every region is a div. Every navigation menu is a div. Every header, every sidebar, every footer is a div.

html

```
<div class="page-header">
  <div class="logo">...</div>
  <div class="navigation">...</div>
</div>
<div class="page-content">
  <div class="article">
    <div class="article-title">...</div>
    <div class="article-body">...</div>
  </div>
  <div class="sidebar">...</div>
</div>
<div class="page-footer">...</div>
```

A page like this renders fine, and CSS can make it look great. But the structure has lost something important:

- **Screen readers** see one undifferentiated stream of divs. They cannot offer "skip to main content" or "jump to navigation" because there is no main content or navigation element to jump to.
- **Search engines** have to guess what each region is from the class names, which they may or may not get right.
- **Browser features** like reader mode rely on knowing where the main content of the page is. With div soup, they fall back to fuzzy heuristics.
- **Future developers** have to read the CSS to figure out what each div is for.



The class names tell us, the people who wrote the page, what each block is. The semantic HTML elements tell everyone else.

Semantic layout elements

HTML provides named container elements for the parts of a page that show up almost everywhere. They look like divs to CSS, but they carry meaning.

Element	Use it for
<header>	Introductory content at the top of a page or section: logo, page title, intro t
<nav>	A major navigation block: primary nav, table of contents, breadcrumbs

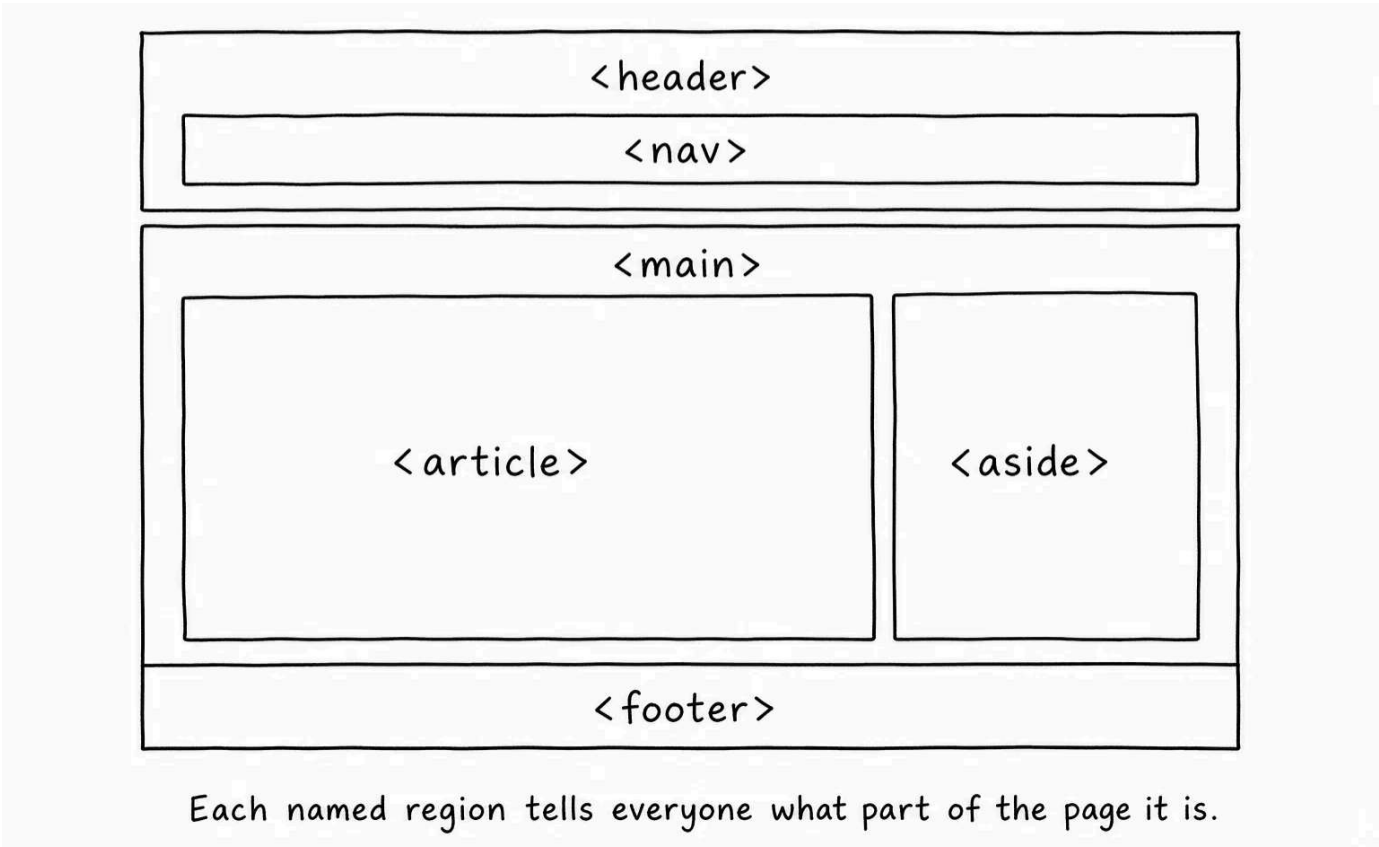
Element	Use it for
<main>	The single main content of the page; used once per page
<section>	A thematic group of content, usually with its own heading
<article>	A standalone, self-contained piece of content: a blog post, a comment, a news item
<aside>	Content related to the surroundings but not part of the main flow: sidebars, pull quotes
<footer>	Closing content for a page or section: copyright, contact, related links

Used together, the same div-soup page above becomes:

html

```
<header>
  
  <nav>
    <a href="/">Home</a>
    <a href="/pricing">Pricing</a>
    <a href="/blog">Blog</a>
  </nav>
</header>
<main>
  <article>
    <h1>The article title</h1>
    <p>The article body...</p>
  </article>
  <aside>
    <h2>Related posts</h2>
    ...
  </aside>
</main>
<footer>
  <p>© 2026 Acme. All rights reserved.</p>
</footer>
```

The layout can still look exactly the same, but every container now tells the browser, the screen reader, and the next developer what its job is. Semantic elements do not replace CSS. CSS still does the visual work. The win is structural.



Picking the right semantic element

A few patterns help when deciding which element to use.

`<main>` appears at most once per page. It holds the part of the page that is unique to that route: not the header, not the footer, not the sidebar. Browsers, screen readers, and reader-mode features all use `<main>` as the anchor for "this is the content the user actually came for".

`<article>` and `<section>` look similar but mean different things. An `<article>` is something that would still make sense on its own if you lifted it out and put it somewhere else: a blog post, a news story, a comment, a product card. A `<section>` is a thematic group of content with its own heading inside a larger document. If you cannot give it a clear heading, it is probably a `<div>`, not a `<section>`.

`<nav>` is for major navigation blocks. A short list of links at the bottom of a paragraph does not need to be wrapped in `<nav>`. Use it for navigation that helps users move around the site as a whole. Too many `<nav>` elements weaken the signal that the important ones carry.

`<aside>` is for content related to the surrounding content but not part of its main flow: callouts in the margin, lists of related posts, pull quotes. It is not just "anything on the side of the page". It is content the page could survive without.

When no semantic element fits, fall back to `<div>` or `<span>`. There is nothing wrong with using them; the goal is to use them only when nothing more specific applies.

💡 **Stay specific** - If you find yourself reaching for `<div>`, pause for a second. Is this a header, a navigation block, a section, an article, an

aside, or a footer? If yes, use the right element. If not, `<div>` is fine.

Why semantic HTML matters

Semantic markup is one of the cheapest improvements you can make to a page. The same code that renders identically in the browser also brings:

- **Accessibility.** Screen readers expose semantic regions as "landmarks" that users can jump between with a single keystroke. "Skip to main content" is a real shortcut in many assistive tools, and `<main>` is what makes it work.
- **SEO and crawlers.** Search engines use semantic structure as a hint for what the page is about. Tools that summarize or excerpt pages do the same.
- **Tooling.** Reader modes, browser extensions, screen capture tools, and translation tools all behave more reliably on semantic markup.
- **Maintenance.** A future developer, often you six months from now, can read the page's HTML and immediately see what each region is for, without digging through the CSS.

None of these wins require any new tags you have not already met. They come from

using `<header>`, `<nav>`, `<main>`, `<section>`, `<article>`, `<aside>`, and `<footer>` instead of a generic `<div>`.

 **Use native HTML before ARIA** - ARIA attributes can add accessibility information when native HTML cannot express something clearly. They are not a replacement for good markup. Prefer real elements like `<button>`, `<nav>`, `<main>`, `<label>`, and `<table>` before adding `role` or `aria-*` attributes. Used incorrectly, ARIA can make a page harder to use.



AI Tutor

What are ARIA attributes? >

When layout comes from app data

Most real pages are not hand-written one file at a time. They are built from templates and components. A few habits keep the semantic structure intact:

- **Pick semantic containers at the component level.** A `Header` component should render a `<header>`, not a `<div className="header">`. The naming and the markup should agree.

- **Keep one main element per page.** Even if your layout has several panes, only one of them should be the page's `<main>` content. Components that just wrap their children should usually use a generic element, not `<main>`.
- **Keep heading levels in sync with the structure.** A `<section>` without a heading is usually a sign you need a `<div>` instead. If the template puts a `<section>` on the page, it should also put a real `<h2>` or `<h3>` inside it.

Try it

Save this as `layout.html` and open it in your browser. It is intentionally written with only `<div>` and `<span>`:

html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Layout practice</title>
  </head>
  <body>
    <div>
      <div>My blog</div>
      <div>
        <a href="/">Home</a>
        <a href="/about">About</a>
      </div>
    </div>
    <div>
      <div>
        <div>An interesting post</div>
        <div>A paragraph of post content.</div>
      </div>
      <div>
        <div>Related</div>
        <div>Links to other posts.</div>
      </div>
    </div>
    <div>© 2026 My blog</div>
  </body>
</html>
```

Now refactor it using the semantic elements from this lesson. Replace the outer top group with `<header>` and put the links inside a `<nav>`. Wrap the body in `<main>`, the post in `<article>`, and the related links in `<aside>`. Wrap the bottom line in `<footer>`. Promote the post title to an `<h1>` and the related-links heading to an `<h2>`.

Open DevTools and look at the **Elements** tab. Click on each region in turn. The browser highlights it on the page and shows its tag name. With the original file, every region was a `div`. With the refactored version, each region now carries its real role.

Knew

0 Items

Learnt

0 Items

Skipped

0 Items

ResetProgress

Test Your Knowledge

What is the difference between block and inline elements?

[Click to Reveal the Answer](#)

Already Know that

Didn't Know that

Skip Question

✓

LESSON COMPLETE

Nicely done.

Up next: Pricing Comparison Table

Next Lesson

https://roadmap.sh/packs/html/containers-and-semantic-layout

8/8